

Verida Protocol (VDA)

SMART CONTRACT

Security Audit

Performed on Contracts:

VeridaDIDLinkage.sol
VeridaDIDRegistry.sol
NameRegistry.sol
RewardToken.sol
VDARewardContract.sol
SoulboundNFT.sol
VDAVerificationContract.sol

Platform

ETH / MATIC

hashlock.com.au

APRIL 2023

Table of Contents

Executive Summary	4
Project Context	4
Verida's Executive Team Leadership	7
Audit scope	8
Security Rating	9
Standardised Checks	10
Intended Smart Contract Functions	12
Function List	14
Code Quality	22
Audit Resources	22
Dependencies	22
Severity Definitions	23
Audit Findings	23
Post Audit Additions	62
Centralisation	63
Owner executable functions	64
Conclusion	65
Our Methodology	66
Disclaimers	68
About Hashlock	69
Appendix	70

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Verida's team partnered with Hashlock to conduct a security audit of their ethereum smart contracts prior to their project's launch. Hashlock manually and proactively reviewed the code in order to ensure the protocols team and community is ready for mainnet deployment.

This audit is the second of two audits we conducted for Verida, the first audit's scope was the Verida token and its functionality, this second audit report's scope is the wider Verida protocol.

Project Context

[Verida Network](#) is a multi-chain protocol for interoperable decentralised data storage, communications infrastructure and authentication solutions built around the fundamentals of individual ownership of data and identity. This network can be interfaced with end users via their mobile app, the [Verida Wallet](#). Their ecosystem consists of a range of products and protocols interacting with each other, and openly invites developers and builders to contribute and add to their personalised data innovation. Users can browse encrypted data and apply a set of delegated permissions when consensually sharing it with other users and services. There is a single ERC20 smart contract for Verida Token, and Verida will use a third party audited token management and distribution platform for the token lockup requirements.

Token Name: Verida Token (VDA)

Contract Address: N/A

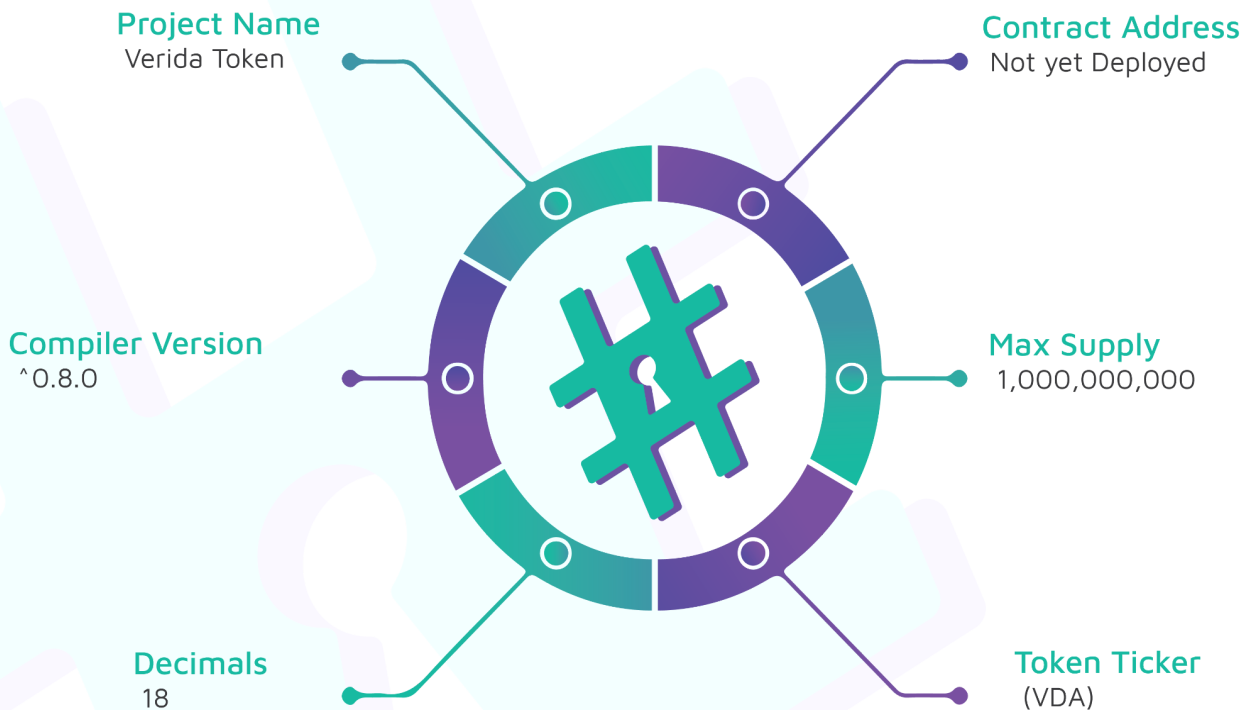
Compiler Version: ^0.8.0

Max Supply: 1,000,000,000

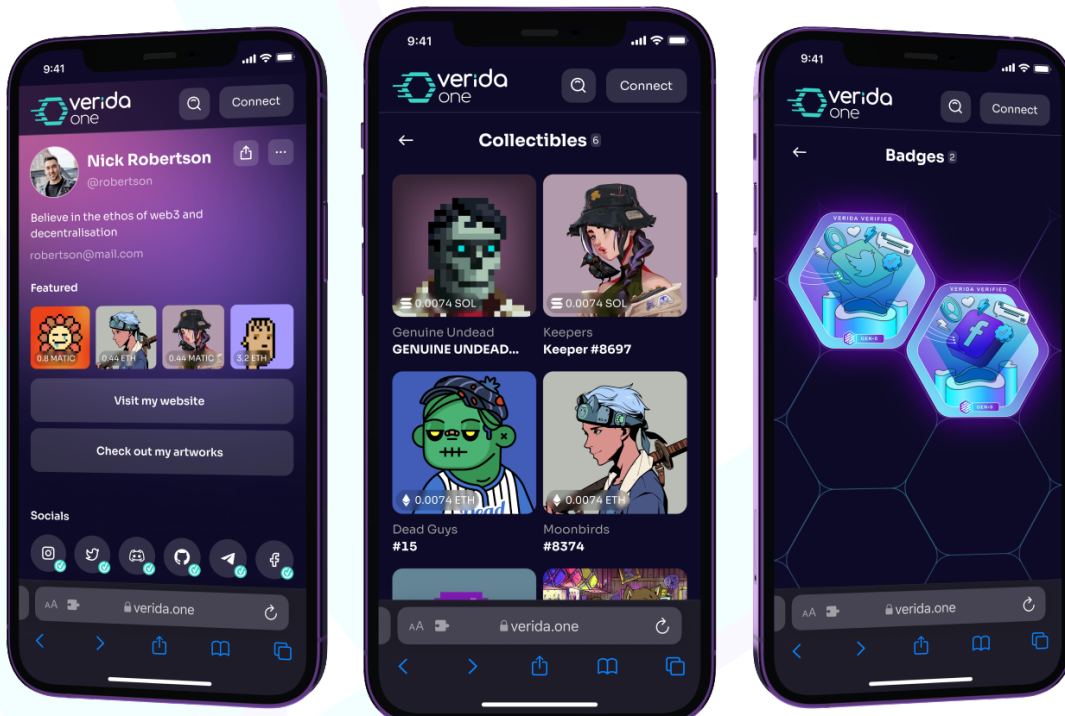
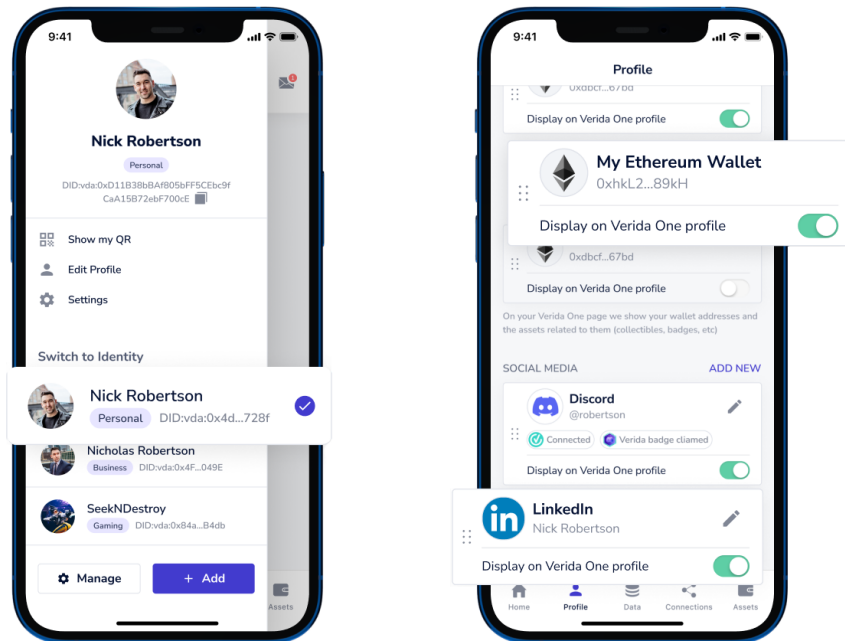
Decimals: 18

Logo:

verida network

Visualised Context (token):

Platform Visualisation:



Verida's Executive Team Leadership

Title	Name	Photo	Links
CEO Co-Founder	Chris Were		Linkedin
COO Co-Founder	Ryan Kris		Linkedin

See their wider team [here](#).

Audit scope

We at Hashlock audited the solidity code within the Verida Protocol, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken through both manual and software assisted analysis.

Description	Project Review and Security Analysis Report for Verida Protocol Smart Contracts and other factors.
Platform	Polygon, Ethereum and Others / Solidity
Audit Date	March 14, 2023
File 1	VeridaDIDLinkage.sol
File 1 Github Commit Hash	e982e2688c681fddadad45b347b27dff905dc770
File 2	VeridaDIDRegistry.sol
File 2 Github Commit Hash	23973b7957e4425623a18f9ecbf0bd0478bf10c1
File 3	NameRegistry.sol
File 3 Github Commit Hash	63f508b0bd5b0f84a6df144b05e2ff2131ebde14
File 4	RewardToken.sol
File 4 Github Commit Hash	7cd19e7e8a9bcd5af5c6a33e3994c345ea3926a0
File 5	VDARewardContract.sol
File 5 Github Commit Hash	7cd19e7e8a9bcd5af5c6a33e3994c345ea3926a0
File 6	SoulboundNFT.sol
File 6 Github Commit Hash	4043e286b290a9e97c20ab52d27dd885b09e9224
File 7	VDAVerificationContract.sol

Security Rating

After our findings were resolved, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified multiple significant vulnerabilities that will need to be addressed before mainnet launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

Hashlock found:

0 Critical severity vulnerabilities

2 High severity vulnerabilities

6 Medium severity vulnerabilities

12 Low and Informational severity vulnerabilities

11 Gas Optimisations

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Reviewed
	Outdated Solidity Version	Reviewed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Reviewed
	Lack of check on input parameters	Reviewed
	Function input parameters check bypass	Passed
	Correct Access control	Reviewed
	Built in emergency features	Passed
	Correct event logs	Reviewed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Reviewed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Reviewed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: FAILED

Revised Audit Result: PASSED

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
File 1 VeridaDIDLinkage.sol Owner can add identifier types.	YES, This is valid.
File 2 VeridaDIDRegistry.sol Owners can get DIDs	YES, This is valid.
File 3 NameRegistry.sol - Owner can add suffixes for names. - Owner can update maximum names for a DID to register.	YES, This is valid.
File 4 RewardToken.sol - Token Name: Verida Reward Token - Token Symbol: Vdar - Decimals: 18 - Maximum Supply: 10 million <u>Ownership Control:</u> - Owner can mint the amount from the address.	YES, This is valid.
File 5 VDARewardContract.sol - Owner can add a claim type. - Owner can remove the claim type. - Owner can update claim type rewards. - Owner can add a trusted address. - Owner can remove the trusted address. - Owner can check whether the address is in trustedAddressList.	YES, This is valid.
File 6 SoulboundNFT.sol Name: Verida SBT	YES, This is valid.

• Symbol: VSBT • Token transfer is blocked. • Openzeppelin library used.	
File 7 VDAVerificationContract.sol • Owner can add a trusted signer address. • Owner can remove a trusted signer's address.	YES, This is valid.

Function List

VeridaDIDLinkage.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	initialize	write	initializer	No Issue
3	addIdentifierType	external	Critical operation lacks event log	Refer Audit Findings
4	isTrustedSignerAddress	external	Passed	No Issue
5	link	external	Passed	No Issue
6	unlink	external	Passed	No Issue
7	isLinked	external	Passed	No Issue
8	lookup	external	Passed	No Issue
9	getLinks	external	Passed	No Issue
10	parseAddr	internal	Passed	No Issue
11	parseIdentifier	internal	Passed	No Issue
12	__VDAVerificationContract_init	internal	access only Initializing	No Issue
13	__VDAVerificationContract_init_unchained	internal	access only Initializing	No Issue
14	addTrustedSigner	external	access only Owner	No Issue
15	removeTrustedSigner	external	access only Owner	No Issue
16	nonce	external	Passed	No Issue
17	verifyData	internal	Passed	No Issue
18	verifyDataWithSigners	internal	Passed	No Issue
19	verifyRequest	internal	Passed	No Issue

VDAVerificationContract.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__VDAVerificationContract_init	internal	access only Initializing	No Issue
3	__VDAVerificationContract_init_unchained	internal	access only Initializing	No Issue
4	addTrustedSigner	external	Critical operation lacks event log	Refer Audit Findings
5	removeTrustedSigner	external	Critical operation lacks event log	Refer Audit Findings
6	nonce	external	Passed	No Issue
7	verifyData	internal	Passed	No Issue
8	verifyDataWithSigners	internal	Passed	No Issue
9	verifyRequest	internal	Passed	No Issue
10	__Ownable_init	internal	access only Initializing	No Issue
11	__Ownable_init_unchained	internal	access only Initializing	No Issue
12	onlyOwner	modifier	Passed	No Issue
13	owner	read	Passed	No Issue
14	_checkOwner	internal	Passed	No Issue
15	renounceOwnership	write	access only Owner	No Issue
16	transferOwnership	write	access only Owner	No Issue
17	_transferOwnership	internal	Passed	No Issue

VeridaDIDRegistry.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initializing	No Issue
3	__Ownable_init_unchain ed	internal	access only Initializing	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue
7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	initialize	write	initializer	No Issue
11	isRegistered	internal	Passed	No Issue
12	register	external	Passed	No Issue
13	revoke	external	Passed	No Issue
14	_getController	internal	Passed	No Issue
15	getController	external	Passed	No Issue
16	setController	external	Passed	No Issue
17	lookup	external	Passed	No Issue
18	nonce	external	Passed	No Issue
19	activeDIDCount	external	Passed	No Issue
20	getDIDs	external	access only Owner	No Issue

NameRegistry.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initializing	No Issue
3	__Ownable_init_unchain ed	internal	access only Initializing	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue
7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	initialize	write	initializer	No Issue
11	nonce	read	Passed	No Issue
12	register	external	Passed	No Issue
13	unregister	external	Passed	No Issue
14	findDID	external	Passed	No Issue
15	getUserNameList	external	Passed	No Issue
16	addSuffix	external	access only Owner	No Issue
17	isValidSuffix	read	Passed	No Issue
18	getSuffix	write	Passed	No Issue
19	isValidCharacter	write	Passed	No Issue
20	updateMaxNamesPerDID	external	access only Owner	No Issue

RewardToken.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initializing	No Issue
3	__Ownable_init_unchain ed	internal	access only Initializing	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue
7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	__RewardToken_init	write	initializer	No Issue
11	__RewardToken_init_unc hained	internal	Passed	No Issue
12	decimals	read	Duplicate function	Refer Audit Findings
13	mint	external	access only Owner	No Issue
14	__ERC20_init	internal	access only Initializing	No Issue
15	__ERC20_init_unchained	internal	access only Initializing	No Issue
16	name	read	Passed	No Issue
17	symbol	read	Passed	No Issue
18	decimals	read	Passed	No Issue
19	totalSupply	read	Passed	No Issue
20	balanceOf	read	Passed	No Issue
21	transfer	write	Passed	No Issue

22	allowance	read	Passed	No Issue
23	approve	write	Passed	No Issue
24	transferFrom	write	Passed	No Issue
25	increaseAllowance	write	Passed	No Issue
26	decreaseAllowance	write	Passed	No Issue
27	_transfer	internal	Passed	No Issue
28	_mint	internal	Passed	No Issue
29	_burn	internal	Passed	No Issue
30	_approve	internal	Passed	No Issue
31	_spendAllowance	internal	Passed	No Issue
32	_beforeTokenTransfer	internal	Passed	No Issue
33	_afterTokenTransfer	internal	Passed	No Issue

VDARewardContract.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	__Ownable_init	internal	access only Initializing	No Issue
3	__Ownable_init_unchained	internal	access only Initializing	No Issue
4	onlyOwner	modifier	Passed	No Issue
5	owner	read	Passed	No Issue
6	_checkOwner	internal	Passed	No Issue
7	renounceOwnership	write	access only Owner	No Issue
8	transferOwnership	write	access only Owner	No Issue
9	_transferOwnership	internal	Passed	No Issue
10	__VDARewardContract_init	write	initializer	No Issue
11	__VDARewardContract_init_unchained	internal	Passed	No Issue
12	getClaimType	external	access only Existing Claim Type	No Issue
13	addClaimType	external	access only Owner	No Issue
14	removeClaimType	external	access only Owner	No Issue
15	updateClaimTypeReward	external	access only Owner	No Issue
16	addTrustedAddress	external	access only Owner	No Issue
17	removeTrustedAddress	external	access only Owner	No Issue
18	isTrustedAddress	external	access only Owner	No Issue
19	claim	external	Critical operation lacks event log	Refer Audit Findings

SoulboundNFT.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	initialize	write	initializer	No Issue
3	tokenURI	read	Passed	No Issue
4	_beforeTokenTransfer	internal	Passed	No Issue
5	_afterTokenTransfer	internal	Passed	No Issue
6	locked	external	Passed	No Issue
7	totalSupply	external	Passed	No Issue
8	getTrustedSignerAddresses	external	Passed	No Issue
9	isValidSBTType	read	Passed	No Issue
10	addSBTType	internal	Passed	No Issue
11	claimSBT	external	Passed	No Issue
12	getClaimedSBTList	external	Passed	No Issue
13	isSBTClaimed	external	Passed	No Issue
14	tokenInfo	external	Passed	No Issue
15	burnSBT	external	Passed	No Issue
16	__VDAVerificationContract_init	internal	access only Initializing	No Issue
17	__VDAVerificationContract_init_unchained	internal	access only Initializing	No Issue
18	addTrustedSigner	external	access only Owner	No Issue
19	removeTrustedSigner	external	access only Owner	No Issue
20	nonce	external	Passed	No Issue
21	verifyData	internal	Passed	No Issue
22	verifyDataWithSigners	internal	Passed	No Issue
23	verifyRequest	internal	Passed	No Issue

Code Quality

This audit scope involves the solidity smart contracts of the Verida project, as outlined in the Audit Scope section. All contracts, libraries and interfaces follow standard best practices and avoid unnecessary complexity that increases the likelihood of exploitation.

All code is well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Verida Protocol's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are very helpful in understanding the overall architecture of the protocol.

We were also provided with project documentation and links to assist with our understanding of the project's architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
Critical	Critical vulnerabilities are of immediate concern, and allow hackers or malicious code to steal funds, take ownership of the contract, or other devastating actions.
High	High vulnerabilities are not attacked as easily, but still have significant implications on the protocol.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Informational	These are mostly preferred factors that won't affect security.

Audit Findings

Critical

No critical vulnerabilities were found.

High

1) Critical inconsistencies between VeridaDIDRegistry.sol and VDAVerificationContract.sol

A trusted signer can be added in VDAVerificationContract.sol as any address, not only DID addresses, and this does not register a DID in the DIDRegistry contract. Therefore trusted signers won't always be in the DID registry.

`_nonce` is in both `VeridaDIDRegistry.sol` and `VDAVerificationContract.sol` but not incremented equally in both. When `_nonce` is incremented in one contract, it is not incremented in the other. The contracts don't use the variable for shared logic but there is no reliable `_nonce` count in the protocol.

VDAVerificationContract.sol:

```

38  /**
39      * @notice Add a trusted signer
40      * @dev Only the contract owner can add
41      * @param didAddress Trusted signer address
42      */
43  function addTrustedSigner(address didAddress) external onlyOwner {
44      require(!_trustedSigners.contains(didAddress), "Already registered");
45      _trustedSigners.add(didAddress);
46  }

```

VeridaDIDRegistry.sol register function:

```

62  function register(address didAddress, string[] calldata endpoints, bytes calldata signature) external override {
63      if (_nonce[didAddress] != 0) {
64          require(isRegistered(didAddress), "Revoked DID address");
65      }

```

Status: Acknowledged - Intended: Verida wants each chain to be able to operate separately on different chains. A trusted signer is not assumed to be a DID.

2) NameRegistry.sol: Potential data corruption caused by updating max names per DID

The `updateMaxNamesPerDID` is currently configured in such a way that it can be lowered. This can cause inconsistencies in the database.

Assume the `MaxNamesPerDID` is set to 3 and User A registers 3 names. The `maxNamesPerDID` is then decreased to 2. User A now has more registered names than the maximum.

This is not now a problem, but if the protocol decides to incorporate a new functionality based on names in the future (e.g., each name can mint x), this could lead to certain users bypassing the max.

Recommendation: Add a check to make sure the new count > `MaxNamesPerDID`

File: `VDA-Name-Registry/contracts/NameRegistry.sol`

```
258     function updateMaxNamesPerDID(uint count) external onlyOwner {
259         require(count > 0, "Zero not allowed");
260         uint orgValue = maxNamesPerDID;
261         maxNamesPerDID = count;
262
263         emit UpdateMaxNamesPerDID(orgValue, count);
264     }
```

Status: Resolved

Medium

1) Lack of critical events throughout the entire protocol

Examples include:

VDAVerificationContract.sol

addTrustedSigner
removeTrustedSigner

VeridaDIDLinkage.sol

addIdentifierType

VDARewardContract.sol

claim

Recommendation: Add critical events. Although these are onlyOwner functions, it will allow us to monitor calls to these functions and see if onlyOwner has been bypassed.

Status: Resolved

2) VeridaDIDRegistry.sol - If statement written incorrectly without curly brackets

```
VeridaDIDRegistry.sol

122     function _getController(address didAddress) internal view returns(address) {
123         if (_DIDInfo[didAddress].controller == address(0x0))
124             return didAddress;
125         return _DIDInfo[didAddress].controller;
126     }
```

Recommendation: If Statement should be:

```
if (_DIDInfo[didAddress].controller == address(0x0)) {
    Return didAddress;
}

Return _DIDInfo[didAddress].controller;
```

Status: Resolved

3) VeridaDIDRegistry.sol - External, irreversible revoking with signature

```
VeridaDIDRegistry.sol

62 function register(address didAddress, string[] calldata endpoints, bytes calldata signature) external override {
63     if (_nonce[didAddress] != 0) {
64         require(isRegistered(didAddress), "Revoked DID address");
65     }
}
```

```
VeridaDIDRegistry.sol

97 function revoke(address didAddress, bytes calldata signature) external override {
98     require(isRegistered(didAddress), "Unregistered address");
99 }
```

If an address is revoked and it has a `_nonce` of 1 or more, it can never be registered again. Revoke is also an external function with signature validation as its safeguard.

Recommendation: Acknowledged - Intended. The Verida team uses permanent personal data deletion as a compliance and selling point of their protocol. They want users to know that they can remove their data if they wish.

4) SoulboundNFT.sol - Insufficient safeguards on tokenId.

Insufficient safeguards on TokenID. TokenID is based on TokenCounter. The variable is set and used in consecutive lines, without any checks.

```
SoulboundNFT.sol

213     _tokenIdCounter.increment();
214     uint256 tokenId = _tokenIdCounter.current();
215     _safeMint(sbtInfo.recipient, tokenId);
```

Recommendation: We suggest setting a variable for lastID, then incrementing `_tokenIdCounter`, then setting a variable called `ThisTokenID` with `_tokenIdCounter.current`, and requiring that `thisTokenID` is greater than `lastID`. This adds an additional layer of protection around the critical variable that is `tokenId`.

Status: **Acknowledged**

5) SoulboundNFT.sol Implementation Errors.

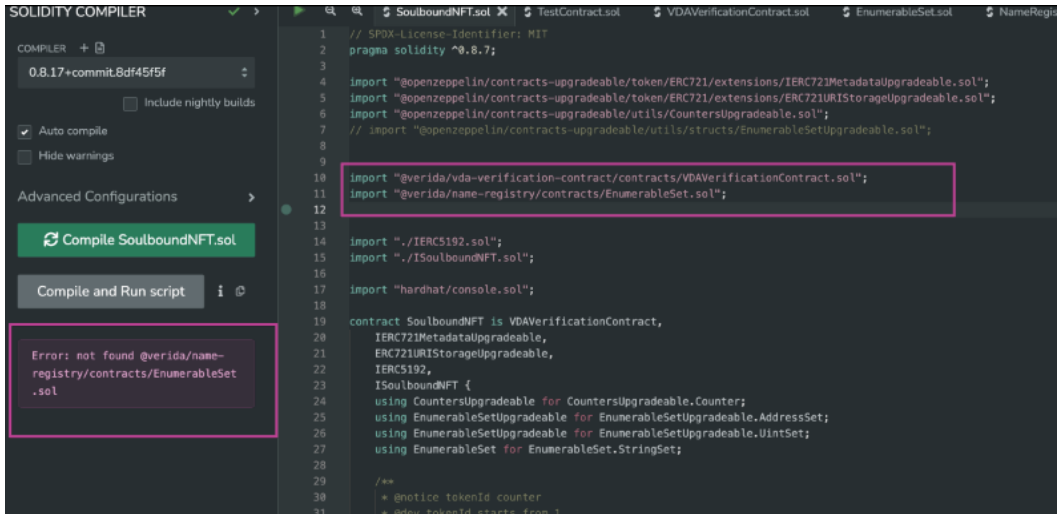
Functions `_beforeTokenTransfer` and `_afterTokenTransfer` result in compiler errors because they do not correctly implement the functions they are trying to override. The functions then use the 'super' keyword to pass parameters to the inherited open zeppelin functions, however those functions expect 4 parameters, not the 3 that were passed. This also causes compilation errors.

```

84  /**
85   * @dev Overrides of super class function. Restrict to be transferred for locked token
86   * See {IERC721Upgradeable}
87   */
88  function _beforeTokenTransfer(
89      address from,
90      address to,
91      uint256 tokenId
92  ) internal override virtual {
93      require(from == address(0) || to == address(0), "Err: token transfer is BLOCKED");
94      super._beforeTokenTransfer(from, to, tokenId);
95  }
96

```

Finally, compiler errors occur as EnumerableSet.sol cant be found via its import.



Recommendation: Correct the compiler errors via correct override and logic.

Status: Resolved

6) SoulboundNFT.sol - SBT Claimed visibility is only available to the SBT owner.

```

252  /**
253   * @dev See {ISoulboundNFT}
254   */
255   function isSBTClaimed(string calldata sbtType, string calldata uniqueId) external view override returns(bool) {
256       return _userInfo[msg.sender][sbtType][uniqueId] > 0;
257   }

```

Only the owner can check if an SBT is claimed because it uses msg.sender, so if the function caller is not the owner, they can't see if it is claimed.

Recommendation: Do not utilise msg.sender in this function.

Status: Resolved

Low / Very Low / Informational / Best practices:

1) VDAVerificationContract.sol, SoulboundNFT.sol and VeridaDIDRegistry.sol: Commented out code

Recommendation: Remove commented out code.

Status: Resolved

2) RewardToken.sol - Duplicate Variables

Name and symbol can be passed directly to `__ERC20_init("Verida Reward Token", "VDAR")`, Decimal variable is already inherited from ERC20Upgradable contract, and can also be directly passed to the MAX_SUPPLY state variable like below.

```
uint256 public constant MAX_SUPPLY = 10_000_000 * (10 ** 18);"
```

Recommendation: Remove or correct duplicate variables.

Status: Resolved - This contract was removed

3) SoulboundNFT.sol - Incorrect ordering on claimSBT and burnSBT

SafeMinting and burning tokens should be the last call before emitting the event in their functions. This is best practice to ensure all other state updates are completed first.

Recommendation: Follow Check, effect, interaction order. Mint and burn at the end of the function after state variables are checked and changed, right before the event is emitted.

Status: Resolved

4) Lack of input validation throughout the entire protocol.

Recommendation: Add input validation and checks.

5) SoulboundNFT.sol Incorrect specification

VDA-SBT README.md says that the SoulboundNFT.sol contract depends on the NameRegistry.sol contract, but it doesn't import or inherit the name registry contract in its code. It only inherits the EnumerableSet.sol contract in the name registry folder.

If this is intended, please update the README.md. Otherwise, inherit and use the NameRegistry.sol contract.

Dependency of Verida contracts

This project depends on the following Verida contracts:

- @verida/vda-verification-base
- @verida/name-registry
- @verida/encryption-utils (For test purpose)

Recommendation: Update README.md file.

Status: resolved

6) Not the most recent solidity version across protocol. The Solidity version is also not consistent across the protocol.

Recommendation: Ensure each contract utilises pragma solidity ^0.8.17 as a best practice.

Status: resolved

7) NameRegistry.sol - Misleading naming in findDID function

```

140      /**
141       * @dev Find did for name
142       * @param name user name. Must be registered
143       * @return DID address of user
144       */
145      function findDID(string memory name) external view returns(address) {
146          name = name.lower();
147
148          address callerDID = _nameToDID[name];
149          require(callerDID != address(0x0), "Unregistered name");
150
151          return callerDID;
152      }
153

```

Uses CallerDID but the name being found doesn't have to be the caller's at all, it can be any name an EOI wants to find. This is misleading.

Recommendation: Update callerDID variable name.

Status: resolved

8) VDAVerificationContract.sol - Spelling mistake in comments

```

70      * Verify any data is signed by a trusted signering DID address

```

Spelling mistake, this should say "signing".

```
VDAVerificationContract.sol

31  /**
32   * @notice Initializer for deploying the contract
33   * @dev Initialize the necessary stuffs those are unique to this contract
34   */
35  function __VDAVerificationContract_init_unchained() internal onlyInitializing {
36  }
```

Should this be “that are unique...”

Recommendation: Fix grammar and spelling mistakes.

Status: Second spelling mistake fixed, first one not fixed yet

9) Possible duplicate endpoints emitted in event

If a user submitted duplicate endpoints in the endpoints parameter, the set would filter them out; however, we use the endpoints parameter when emitting the Register event, therefore duplicates will still appear in the emitted event. This is inconvenient for any off chain protocols that are tracking the protocol.

Recommendation: When emitting the Register event, use the most recent list. We won't have any duplicates this way.

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
VeridaDIDRegistry.sol

62 function register(address didAddress, string[] calldata endpoints, bytes calldata signature ) external override {
63     if (_nonce[didAddress] != 0) {
64         require(isRegistered(didAddress), "Revoked DID address");
65     }
66
67     {
68         bytes memory rawMsg = abi.encodePacked(didAddress, "/");
69         for (uint i = 0; i < endpoints.length; i++) {
70             rawMsg = abi.encodePacked(rawMsg, endpoints[i], "/");
71         }
72
73         rawMsg = abi.encodePacked(rawMsg, _nonce[didAddress]);
74
75         require(VeridaDataVerificationLib.validateSignature(rawMsg, signature, didAddress), "Invalid signature");
76         _nonce[didAddress]++;
77     }
78
79     EnumerableSet.StringSet storage list = _DIDInfo[didAddress].endpoints;
80     list.clear();
81
82     for (uint i = 0; i < endpoints.length; i++) {
83         list.add(endpoints[i]);
84     }
85
86     // Add didAddress for only new registers
87     if (!isRegistered(didAddress)) {
88         _registeredDIDs.add(didAddress);
89     }
90
91     emit Register(didAddress, endpoints);
92 }
```

Status: Acknowledged

10) Lack of event for adding Trusted Signers

TrustedSigners could be something that is monitored/tracked by an off-chain protocol, emitting an event when it is updated so it is easier to track.

There are 2 instances of this issue

File: VDA-Verification-base/contracts/

```
VDAVerificationContract.sol

38  /**
39   * @notice Add a trusted signer
40   * @dev Only the contract owner can add
41   * @param didAddress Trusted signer address
42   */
43  function addTrustedSigner(address didAddress) external onlyOwner {
44      require(!_trustedSigners.contains(didAddress), "Already registered");
45      _trustedSigners.add(didAddress);
46  }
```

File: VDA-Verification-base/contracts/

```
VDAVerificationContract.sol

48  /**
49   * @notice Remove a trusted signer
50   * @dev Only the contract owner can remove
51   * @param didAddress Trusted signer address
52   */
53  function removeTrustedSigner(address didAddress) external onlyOwner {
54      require(_trustedSigners.contains(didAddress), "Unregistered address");
55      _trustedSigners.remove(didAddress);
56  }
```

Recommendation: Emit events for add and remove trusted signers.

Status: resolved

11) Lack of check on maxNamesPerDID (Related to medium vulnerability no.1)

If the owner unintentionally entered the same value for maxNamesPerDID as the current count, nothing would happen and gas would be wasted.

File: VDA-Name-Registry/contracts/NameRegistry.sol

```

258     function updateMaxNamesPerDID(uint count) external onlyOwner {
259         require(count > 0, "Zero not allowed");
260         uint orgValue = maxNamesPerDID;
261         maxNamesPerDID = count;
262
263         emit UpdateMaxNamesPerDID(orgValue, count);
264     }

```

Recommendation: Add `require(count != maxNamesPerDID)`

Status: resolved

12) Lack of event emission after critical initialize() functions

Recommendation: To record the init parameters for off-chain monitoring and transparency reasons, please consider emitting an event after the initialize() functions

Status: Acknowledged

Gas Optimisation:

1) Cache state variable outside loop to save gas

There are 2 instances of this issue

File: VDA-SBT/contracts/SoulBoundNFT.sol

```
test.sol

4      function getTrustedSignerAddresses()
5          external
6          view
7          override
8          returns (address[] memory)
9      {
10         uint length = _trustedSigners.length();
11         address[] memory list = new address[](length);
12         for (uint i = 0; i < length; i++) {
13             // @audit reported
14             list[i] = _trustedSigners.at(i);
15         }
16
17         return list;
18     }
```

Status: Acknowledged

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
test.sol

1      function getDIDs(
2          uint startIndex,
3          uint count
4      ) external view override onlyOwner returns (address[] memory) {
5          // @audit possible gas savings
6          require(
7              count > 0 && (startIndex + count) <= _registeredDIDs.length(),
8              "Out of range"
9          ); // @audit reported
10
11         address[] memory ret = new address[](count);
12         // @audit possible gas savings
13         for (uint i = 0; i < count; i++) {
14             // @audit reported
15             ret[i] = _registeredDIDs.at(startIndex + i);
16         }
17
18         return ret;
19     }
20
21 </code></pre>
22
```

Status: Resolved

2) Don't initialise variables with the default value

Initialising a variable with its default value costs unnecessary gas.

There are 26 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

```
test.sol  
  
1 215   for (uint i = 0; i < length; i++) {  
2  
3 233   uint160 iaddr = 0;  
4  
5 276   uint index = 0;  
6  
7 277   uint8 sepCount = 0;
```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
test.sol  
  
1 84    for (uint i = 0; i < endpoints.length; i++) {  
2  
3 104   for (uint i = 0; i < endpoints.length; i++) {  
4  
5 235   for (uint i = 0; i < length; i++) {  
6  
7 271   for (uint i = 0; i < count; i++) {
```

File: VDA-Name-Registry/contracts/NameRegistry.sol test.sol

```
1 198     for (uint i = 0; i < length; i++) {  
2  
3 249     uint index = 0;  
4  
5 250     uint8 dotCount = 0;
```

File: VDA-SBT/contracts/SoulBoundNFT.sol test.sol

```
1 138     for (uint i = 0; i < length; i++) {  
2  
3 170     for (uint i = 0; i < charSet.length && isValid; i++) {  
4  
5 273     for (uint i = 0; i < length; i++) {
```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```

test.sol

1  403    uint cnt = 0;
2
3  404    for(uint i = 0; i < serviceIdList.length(); i++) {
4
5  417    for(uint i = 0; i < cnt; i++)
6
7  441    uint256 amount = 0;
8
9  444    for(uint i = 0; i < _service.connectedAccounts.length(); i++){
10
11 489    for(uint i = 0; i < length; i++) {
12
13 565    for(uint i = 0; i < length; i++){

```

File: VDA-Verification-base/contracts/VDAVerificationContract.sol

```

test.sol

1  89      bool isVerified = false;
2
3  90      uint index = 0;
4
5  140     bool isVerified = false;
6
7  142     uint index = 0;

```

Status: Resolved

3) Prefix increments cheaper than Postfix increments

`++i` costs less gas than `i++`, especially when it's used in for-loops (`--i/i--` too) Saves 5 gas PER LOOP

There are 15 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

test.sol

```
1 215    for (uint i = 0; i < length; i++) {
```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

test.sol

```
1 84      for (uint i = 0; i < endpoints.length; i++) {
2
3 104     for (uint i = 0; i < endpoints.length; i++) {
4
5 235     for (uint i = 0; i < length; i++) {
6
7 271     for (uint i = 0; i < count; i++) {
```

File: VDA-Name-Registry/contracts/NameRegistry.sol

```

test.sol
1 198     for (uint i = 0; i < length; i++) {

```

File: VDA-SBT/contracts/SoulBoundNFT.sol

```

test.sol
1 138     for (uint i = 0; i < length; i++) {
2
3 170     for (uint i = 0; i < charSet.length && isValid; i++) {
4
5 273     for (uint i = 0; i < length; i++) {

```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```

test.sol
1 404     for(uint i = 0; i < serviceIdList.length(); i++) {
2
3 417     for(uint i = 0; i < cnt; i++)
4
5 444     for(uint i = 0; i < _service.connectedAccounts.length(); i++){
6
7 489     for(uint i = 0; i < length; i++) {
8
9 565     for(uint i = 0; i < length; i++){

```

Status: Resolved

4) Using `> 0` costs more gas than `!= 0` when used on a uint in a `require()` statement**

When dealing with unsigned integer types, comparisons with `!= 0` are cheaper than with `> 0`. This change saves 6 gas per instance

There are 15 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

```
test.sol  
  
1 272    require(strBytes.length > 0, "Invalid identifier");  
2  
3 291    require(...sepPos > 0..., "Invalid identifier");
```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
test.sol  
  
1 268    require(count > 0 && (startIndex + count) <= _registeredDIDs.length(), "Out of range");
```

File: VDA-Name-Registry/contracts/NameRegistry.sol

```
test.sol  
  
1 194     require(length > 0, "No registered DID");  
2  
3 295     require(count > 0, "Zero not allowed");
```

File: VDA-Reward-Contract/contracts/VDARewardContract.sol

```
test.sol  
  
1 23      claimType.reward > 0 && bytes(claimType.schema).length > 0,  
2  
3 66      require(bytes(typeId).length > 0, "Invalid id");  
4  
5 67      require(rewardAmount > 0, "Invalid reward amount");  
6  
7 68      require(bytes(schema).length > 0, "Invalid schema");  
8  
9 98      require(amount > 0, "Invalid reward amount");
```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

test.sol

```
1 216     require(numCredit > 0, "Value cannot be zero");
```

File: VDA-Verification-base/contracts/

test.sol

```
1 81     require(_trustedSigners.length() > 0, "No signers provided");
```

```
2
```

```
3 129    require(validSigners.length > 0, "No signers provided");
```

Status: Resolved

5) $\langle X \rangle += \langle Y \rangle$ COSTS MORE GAS THAN $\langle X \rangle = \langle X \rangle + \langle Y \rangle$ FOR STATE VARIABLES (for state variables ($\neq$ too))

Using the addition/subtraction operator instead of plus-equals saves 113 gas.

There are 7 instances of this issue

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```
test.sol
1 199 creditAmount[identity] += numCredit;
2
3 383 creditAmount[identity] += _service.creditAmount;
4
5 461 creditAmount[identity] += price;
6
7 564 creditAmount[identity] += amount;
8
9 225 creditAmount[identity] -= numCredit;
10
11 261 creditAmount[identity] -= registerPrice;
12
13 422 creditAmount[identity] -= price;
```

Status: Resolved

6) ++i/i++ should be unchecked{++i}/unchecked{i++} when it is not possible for them to overflow, for example when used in for- and while-loops

++i/i++ should be unchecked{++i}/unchecked{i++} when it is not possible for them to overflow, for example when used in for- and while-loops

The unchecked keyword is new in solidity version 0.8.0, so this only applies to that version or higher, which these instances are. This saves 30-40 gas per loop

There are 26 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

```
test.sol
1 215    for (uint i = 0; i < length; i++) {
2
3 283    sepCount++;
4
5 285    index++;
6
7 297    for (index = 0; index < sepPos; index++) {
8
9 302    for (index = sepPos + 1; index < len; index++) {
```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol test.sol

```
1 84      for (uint i = 0; i < endpoints.length; i++) {  
2  
3 105     for (uint i = 0; i < endpoints.length; i++) {  
4  
5 237     for (uint i = 0; i < length; i++) {  
6  
7 274     for (uint i = 0; i < count; i++) {
```

File: VDA-Name-Registry/contracts/NameRegistry.sol test.sol

```
1 198     for (uint i = 0; i < length; i++) {  
2  
3 258     dotCount++;  
4  
5 261     index++;  
6  
7 274     for (index = startIndex; index < len; index++) {
```

File: VDA-SBT/contracts/SoulBoundNFT.sol

```

test.sol

1 151    for (uint i = 0; i < length; i++) {
2
3 170    for (uint i = 0; i < charSet.length && isValid; i++) {
4
5 274    for (uint i = 0; i < length; i++) {

```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

// check other instances where max is capped by require

```

test.sol

1 485    for (uint i = 0; i < serviceIdList.length(); i++) {
2
3 500    cnt = cnt + 1;
4
5 507    i++
6
7 541    for (uint i = 0; i < _service.connectedAccounts.length(); i++) {
8
9 599    for (uint i = 0; i < length; i++) {
10
11 692    for (uint i = 0; i < length; i++) {

```

File: VDA-Verification-base/contracts/

```

test.sol

1 109    index++;
2
3 164    index++;
4
5 203    _nonce[did]++;

```

Status: Resolved

7) Use custom errors rather than revert()/require() strings to save gas (check if only for repeated ones)**

Custom errors are available from solidity version 0.8.4. Custom errors save ~50 gas each time by avoiding having to allocate and store the revert string. Not defining the strings also save deployment gas

There are 84 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

```

1  89      require(_identifierType[kind] != SignerType.Unregistered,"Invalid identifier type");
2
3  99      require(bytes(_identifierController[info.identifier]).length == 0,"Identity already exists");
4
5  163     require(_didIdentifiers[strDID].contains(identifier),"Identifier not linked");
6
7  231     require(tmp.length == 42, "Invalid address");
8
9  272     require(strBytes.length > 0, "Invalid identifier");
10
11 291     require(index == len && sepCount == 1 && sepPos > 0 && sepPos < (len - 1), "Invalid identifier");
12

```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```

1  78      require(isRegistered(didAddress), "Revoked DID address");
2
3  91      require(VeridaDataVerificationLib.validateSignature(rawMsg,signature,didAddress),"Invalid signature");
4
5  129     require(isRegistered(didAddress), "Unregistered address");
6
7  138     require(VeridaDataVerificationLib.validateSignature(rawMsg,signature,controller),"Invalid signature");
8
9  197     require(isRegistered(didAddress), "Unregistered address");
10
11 208     require(VeridaDataVerificationLib.validateSignature(rawMsg,signature,oldController),"Invalid signature");
12
13 230     require(isRegistered(didAddress), "Unregistered address");
14
15 267     require(count > 0 && (startIndex + count) <= _registeredDIDs.length(),"Out of range");

```

File: VDA-Name-Registry/contracts/NameRegistry.sol

```

1  79      require(did != address(0x0), "Invalid zero address");
2  80      require(isValidSuffix(name), "Invalid suffix");
3  86      require(VeridaDataVerificationLib.validateSignature(paramData,signature,did),"Invalid Signature");
4  98      require(_nameToDID[_name] == address(0x0), "Name already registered");
5  105     require(didUserNameList.length() < maxNamesPerDID,"DID can not support any more names");
6  131     require(did != address(0x0), "Invalid zero address");
7  137     require(VeridaDataVerificationLib.validateSignature(paramData,signature,did),"Invalid Signature");
8  151     require(callerDID != address(0x0), "Unregistered name");
9  153     require(callerDID == did, "Invalid DID");
10 177     require(callerDID != address(0x0), "Unregistered name");
11 194     require(length > 0, "No registered DID");
12 216     require(!suffixList.contains(suffix), "Already registered");
13 245     require(nameBytes.length > 0, "No Suffix");
14 263     require(dotCount < 2 && index == len,"Invalid character specified in name");
15 267     require(startIndex < len, "No Suffix");
16 270     require(startIndex > 2 && startIndex < 34, "Invalid name length");
17 295     require(count > 0, "Zero not allowed");

```

File: VDA-Reward-Contract/contracts/VDARewardContract.sol

```

1  66      require(bytes(typeId).length > 0, "Invalid id");
2
3  67      require(rewardAmount > 0, "Invalid reward amount");
4
5  68      require(bytes(schema).length > 0, "Invalid schema");
6
7  70      require(claimType.reward == 0 && bytes(claimType.schema).length == 0,"Already existing ClaimType");
8
9  98      require(amount > 0, "Invalid reward amount");
10
11 117     require(!claims[rawMsg], "Already claimed");
12
13 121     require(rewardToken.balanceOf(address(this)) >= claimType.reward,"Insufficient token in contract");

```

File: VDA-SBT/contracts/SoulBoundNFT.sol

```

1  98      require(from == address(0) || to == address(0),"Err: token transfer is BLOCKED");
2
3  200     require(isValidSBTType(sbtInfo.sbtType), "Invalid SBT type");
4
5  228     require(_userInfo[sbtInfo.recipient][sbtInfo.sbtType][sbtInfo.uniqueId] == 0,"Already claimed type");
6
7  307     require(msg.sender == tokenOwner || msg.sender == owner(),"Invalid operation");

```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```

1 105   require(identity != address(0), "Identity address cannot be zero");
2 109   require(signature.equal(rightSign), "bad_actor");
3 197   require(success, "Token transfer failed");
4 216   require(numCredit > 0, "Value cannot be zero");
5 217   require(creditAmount[identity] >= numCredit, "Not enough credit to remove");
6 223   require(success, "Token transfer failed");
7 249   require(!isRegistered, "Service has already registered!");
8 257   require(creditAmount[identity] >= registerPrice, "Not enough credit to register service");
9 348   require(!isPending, "Service is pending removal");
10 378   require((serviceInfos[serviceId].status == ServiceStatus.PendingRemoval) && (block.timestamp > serviceInfos[serviceId].expireTime),
11 "Not ready to remove");
12 404   require(!isPendingRemoval, "Service is pending removal");
13 405   require(!_service.connectedAccounts.contains(identity), "Already connected");
14 409   require(_service.serviceDetail.maxAccounts > _service.connectedAccounts.length(), "Service hits maximum number of connected accounts");
15 418   require(creditAmount[identity] >= price, "Not enough VDA to connect service");
16 453   require(isConnected, "Account is not connected to service");
17 534   require(block.timestamp >= _service.lastClaimTime + 7 days, "Should claim after 7 days from the last claim");
18 557   require(_service.creditAmount >= amount, "Cannot claim, not enough credit");
19 578   require(!isTimeLimited && isConnected, "Unable to use service");

```

File: VDA-Verification-base/contracts/

```

1 45   require(!_trustedSigners.contains(didAddress), "Already registered");
2
3 55   require(_trustedSigners.contains(didAddress), "Unregistered address");
4
5 81   require(_trustedSigners.length() > 0, "No signers provided");
6
7 112   require(isVerified, "Data is not signed by a valid signing DID");
8
9 129   require(validSigners.length > 0, "No signers provided");
10
11 167   require(isVerified, "Data is not signed by a valid signing DID");

```

Status: **Resolved**

8) Splitting require() statements that use && saves gas

Saves 16 gas per instance.

There are 6 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

```
test.sol  
1 291     require(index == len && sepCount == 1 && sepPos > 0 && sepPos < (len - 1), "Invalid identifier");
```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
test.sol  
1 267     require(count > 0 && (startIndex + count) <= _registeredDIDs.length(), "Out of range");
```

File: VDA-Name-Registry/contracts/NameRegistry.sol

```
test.sol  
1 263     require(dotCount < 2 && index == len, "Invalid character specified in name");  
2  
3 270     require(startIndex > 2 && startIndex < 34, "Invalid name length");
```

File: VDA-Reward-Contract/contracts/VDARewardContract.sol

```
test.sol  
1 70      require(claimType.reward == 0 && bytes(claimType.schema).length == 0,"Already existing ClaimType");
```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```
ServiceRegistry copy.sol  
323 require((serviceInfos[serviceId].status == ServiceStatus.PendingRemoval) && (block.timestamp > serviceInfos[serviceId].expireTime), "Not ready to remove");
```

Status: Resolved

9) Use assembly to check for address(0)

Saves 6 gas per instance if using assembly to check for address(0)

```
test.sol

1 //Example
2 assembly {
3     if iszero(_addr) {
4         mstore(0x00, "zero address")
5         revert(0x00, 0x20)
6     }
7 }
```

There are 12 instances of this issue

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
test.sol

1 176     if (_DIDInfo[didAddress].controller == address(0x0)) return didAddress;
```

File: VDA-Name-Registry/contracts/NameRegistry.sol

```

● ● ● test.sol

1  79      require(did != address(0x0), "Invalid zero address");
2
3  98      require(_nameToDID[_name] == address(0x0), "Name already registered");
4
5  131     require(did != address(0x0), "Invalid zero address");
6
7  151     require(callerDID != address(0x0), "Unregistered name");
8
9  177     require(callerDID != address(0x0), "Unregistered name");

```

File: VDA-Reward-Contract/contracts/VDARewardContract.sol

```

● ● ● test.sol

1  114     if (to == address(0x0)) {
2
3  127     if (owner == address(0x0)) {

```

File: VDA-SBT/contracts/SoulBoundNFT.sol

```

● ● ● test.sol

1  99      require(from == address(0) || to == address(0), "Err: token transfer is BLOCKED");
2
3  114     if (to == address(0x0)) {
4
5  127     if (owner == address(0x0)) {

```

Status: Resolved

10) Functions guaranteed to revert when called by normal users can be marked payable

If a function modifier such as `onlyOwner` is used, the function will revert if a normal user tries to pay the function. Marking the function as payable will lower the gas cost for legitimate callers because the compiler will not include checks for whether a payment was provided.

There are 25 instances of this issue

File: VDA-DID-Linkage/contracts/VeridaDIDLinkage.sol

```
VeridaDIDLinkage.sol
57     function addIdentifierType(string calldata identifierTypeId, bool isSelfSigner) external onlyOwner {
```

File: VDA-DID-Registry/contracts/VeridaDIDRegistry.sol

```
test.sol
1 262     function getDIDs(uint startIndex,uint count) external view override onlyOwner returns (address[] memory) {
```

File: VDA-Reward-Contract/contracts/VDARewardContract.sol

```
test.sol
1 61     function addClaimType(string calldata typeId,uint rewardAmount,string calldata schema) external onlyOwner {
2
3 83     function removeClaimType(string calldata typeId) external onlyOwner onlyExistingClaimType(typeId) {
4
5 94     function updateClaimTypeReward(string calldata typeId,uint amount) external onlyOwner onlyExistingClaimType(typeId) {
6
```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```
test.sol
1 170 function setToken(address _vdaToken) external onlyOwner {
2
3 718 function setVdaPerAccount(string memory infraType,uint256 amount) external onlyOwner {
4
5 731 function setTypeLookup(string memory serviceType,string memory infraType) external onlyOwner {
6
7 738 function setDeregisterDelayDays(uint256 delay) external onlyOwner {
8
9 744 function setMinimumDaysCreditPerService(uint256 minDays) external onlyOwner {
10
11 748 function setPriceChangeDelayDays(uint256 delay) external onlyOwner {
```

File: VDA-Verification-base/contracts/

```
test.sol
1 44 function addTrustedSigner(address didAddress) external onlyOwner {
2
3 54 function removeTrustedSigner(address didAddress) external onlyOwner {
```

Status: Resolved

11) require()/revert() strings longer than 32 bytes cost extra gas

Each extra memory word of bytes past the original 32 incurs an MSTORE which costs 3 gas

There are 9 instances of this issue

File: VDA-Name-Registry/contracts/NameRegistry.sol

```
test.sol
1 105     require(didUserNameList.length() < maxNamesPerDID,"DID can not support any more names");
2
3 263     require(dotCount < 2 && index == len,"Invalid character specified in name");
```

File: VDA-Service-Registry/contracts/ServiceRegistry.sol

```
test.sol
1 257     require(creditAmount[identity] >= registerPrice,"Not enough credit to register service");
2
3 409     require(_service.serviceDetail.maxAccounts > _service.connectedAccounts.length(),"Service hits maximum number of connected accounts");
4
5 418     require(creditAmount[identity] >= price,"Not enough VDA to connect service");
6
7 453     require(isConnected, "Account is not connected to service");
8
9 534     require(block.timestamp >= _service.lastClaimTime + 7 days,"Should claim after 7 days from the last claim");
10
```

File: VDA-Verification-base/contracts/

```
test.sol
1 112     require(isVerified, "Data is not signed by a valid signing DID");
2
3 167     require(isVerified, "Data is not signed by a valid signing DID");
```

Status: **Resolved**

Post Audit Additions

Hashlock reviewed all findings and the relevant changes made that were applicable to our suggestions.

One suggestion was to use assembly for address(0) checks.

Assembly can be highly custom and complex, and the assembly in the revised code was subject to the revision check but not subject to a complete audit process.

Centralisation

The Verida protocol values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on private key security, and internal team responsibility.

There are multiple smart contracts with different objectives around centralization / decentralisation in this protocol. For instance, the reward contract is designed so the team can distribute reward tokens to users and is centralised by intent, whereas the DID Registry is designed to be totally decentralised — however Verida retains the ability to upgrade it in the event there are critical issues identified.



Centralised

Decentralised

Verida is tackling a complex set of technical challenges to provide a unified, trusted, personal data network. Verida is taking a staged approach to decentralisation with an intention to completely decentralise the technology in the future.

Owner executable functions

VDAVerificationContract.sol

- addTrustedSigner: Owner can add a trusted signer address.
- removeTrustedSigner: Owner can remove a trusted signer's address.

NameRegistry.sol

- addSuffix: Owner can add suffixes for names.
- updateMaxNamesPerDID: Owner can update maximum names for a DID to register.

RewardToken.sol

- mint: Owner can mint the amount from the address.

VDARewardContract.sol

- addClaimType: Owner can add a claim type.
- removeClaimType: Owner can remove the claim type.
- updateClaimTypeReward: Owner can update claim type rewards.
- addTrustedAddress: Owner can add a trusted address.
- removeTrustedAddress: Owner can remove the trusted address.
- isTrustedAddress: Owner can check whether the address is in trustedAddressList.

VeridaDIDLinkage.sol

- addIdentifierTypes: Owner can add an identifier type.

VeridaDIDRegistry.sol

- getDIDs: Owners can get DIDs.

Conclusion

After Hashlocks analysis, the Verida protocol seems to have a sound and well tested code base, pending the resolution of our findings. Overall, most of the code is correctly ordered and follows industry best practices.

The project does sacrifice decentralisation for security, as there is a large amount of access control, as well as the manual release of rewards. Hashlock believes this is a sound decision.

As our findings have now been resolved, Hashlock believes Verida's solidity code base is ready for deployment.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

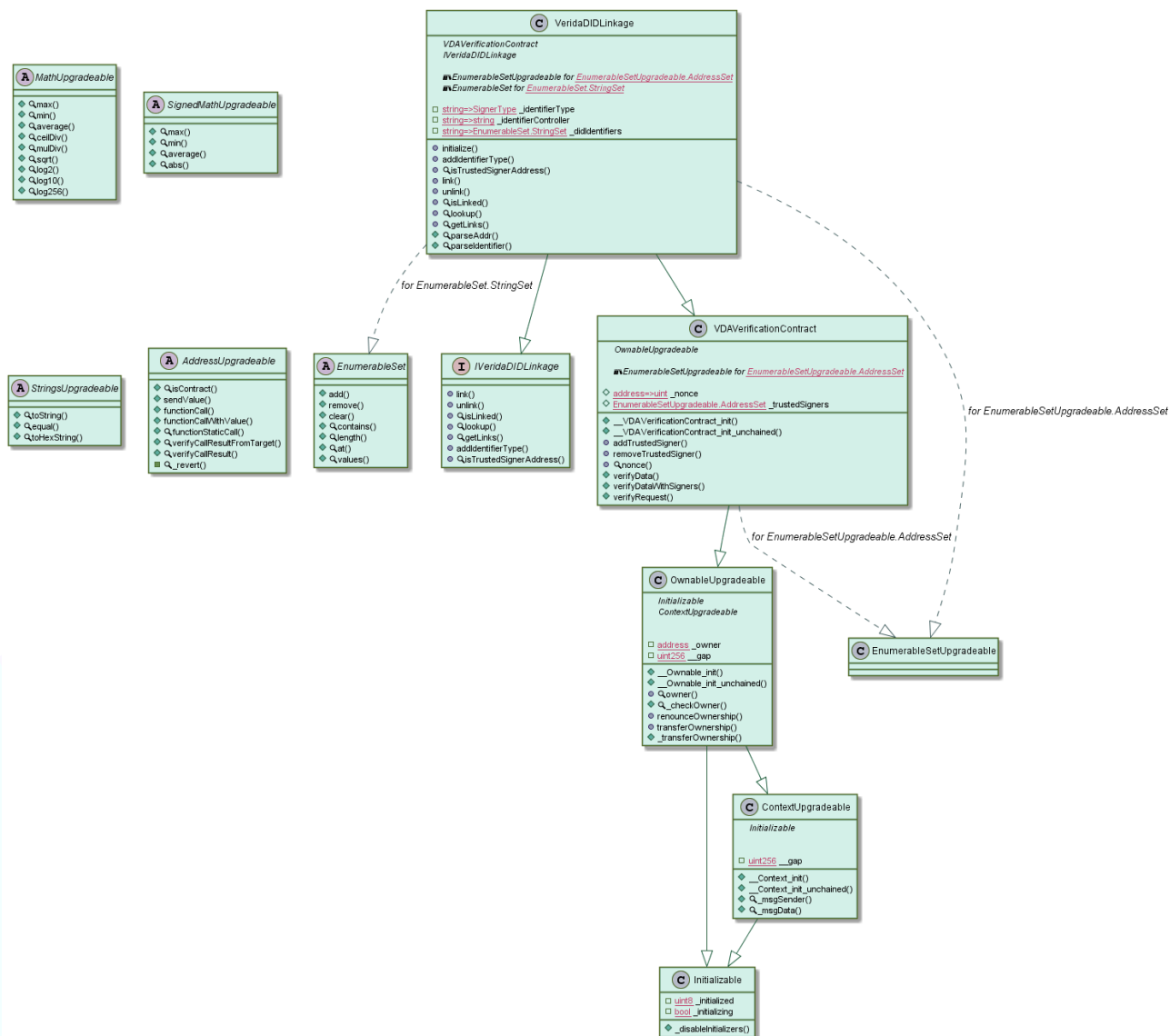
Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

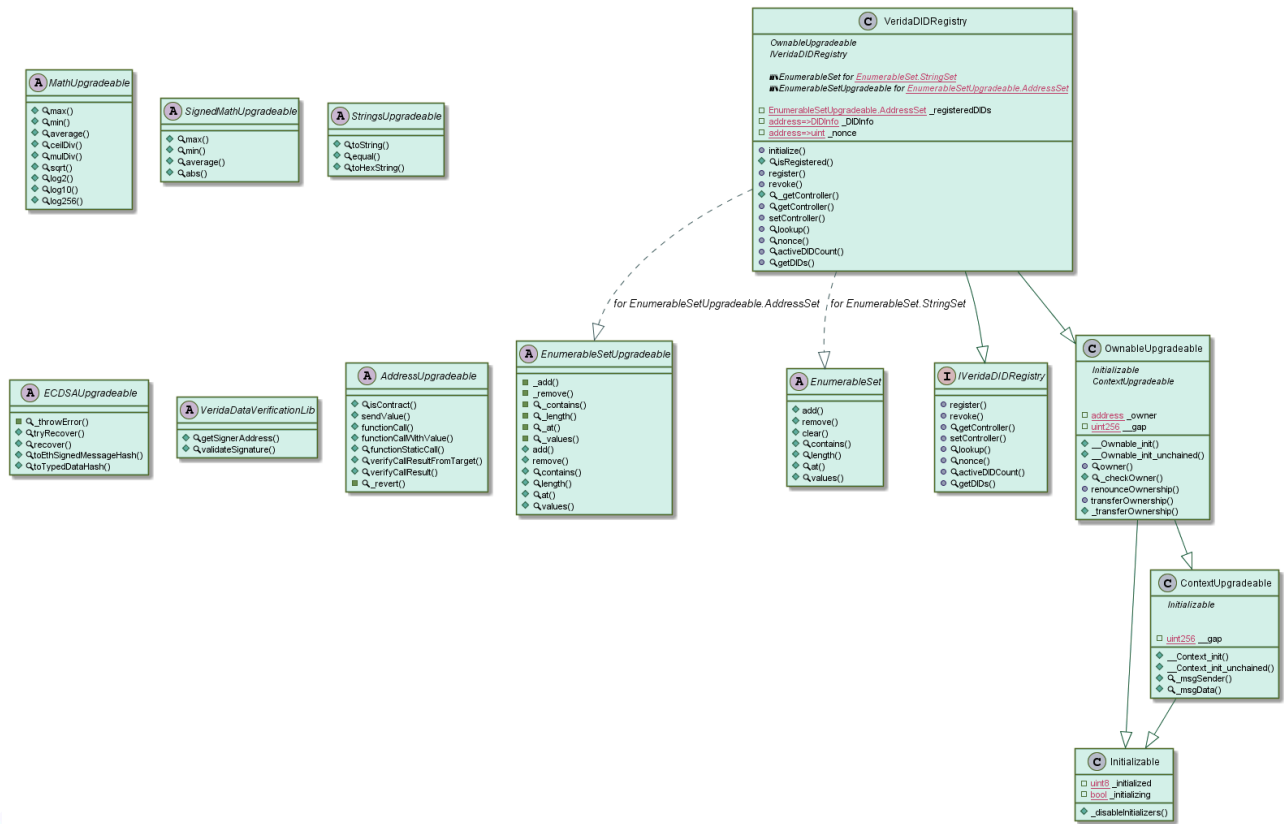
Contact: info@hashlock.com.au

Appendix

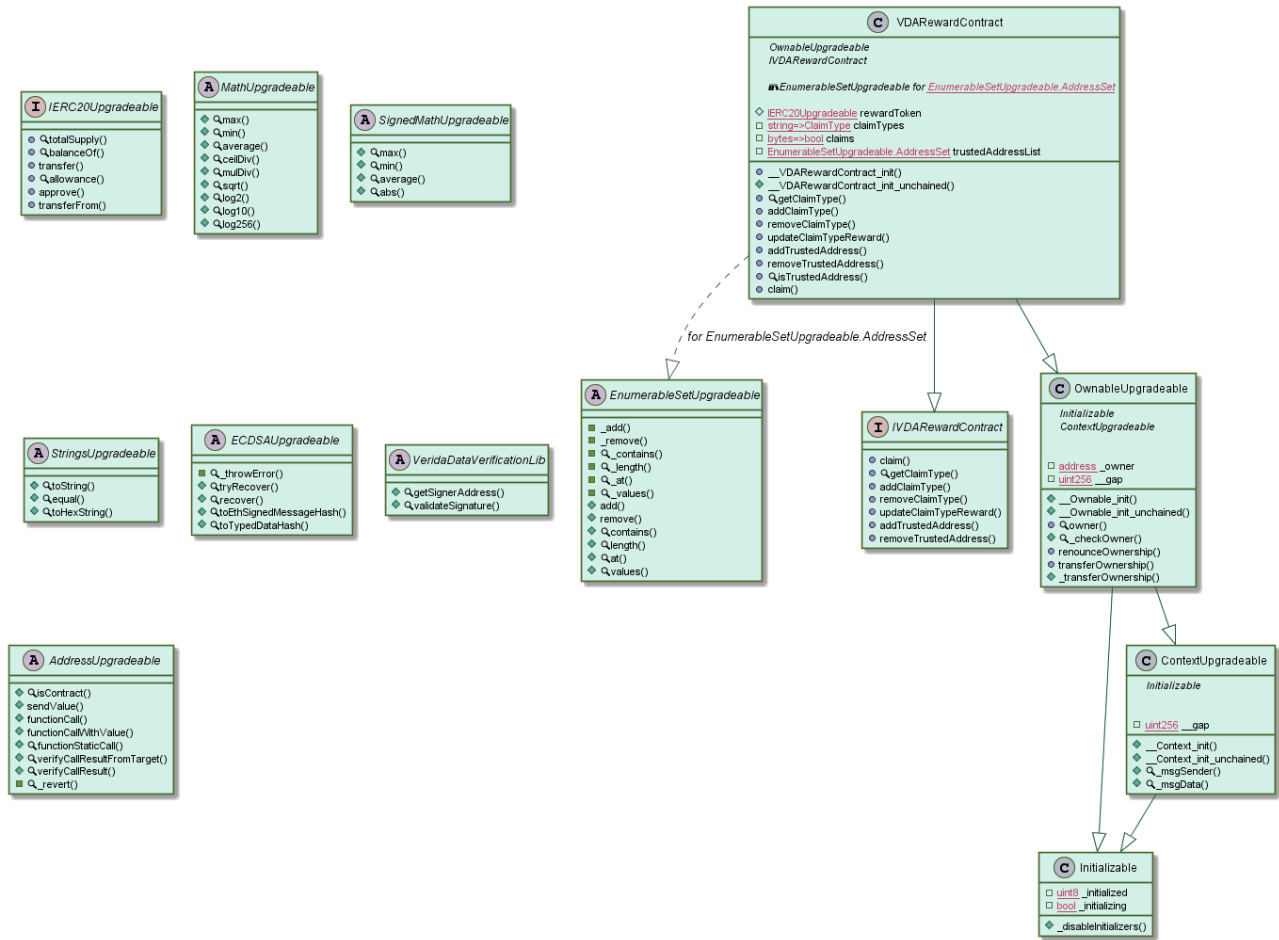
VeridaDIDLinkage Diagram



VeridaDIDRegistry Diagram



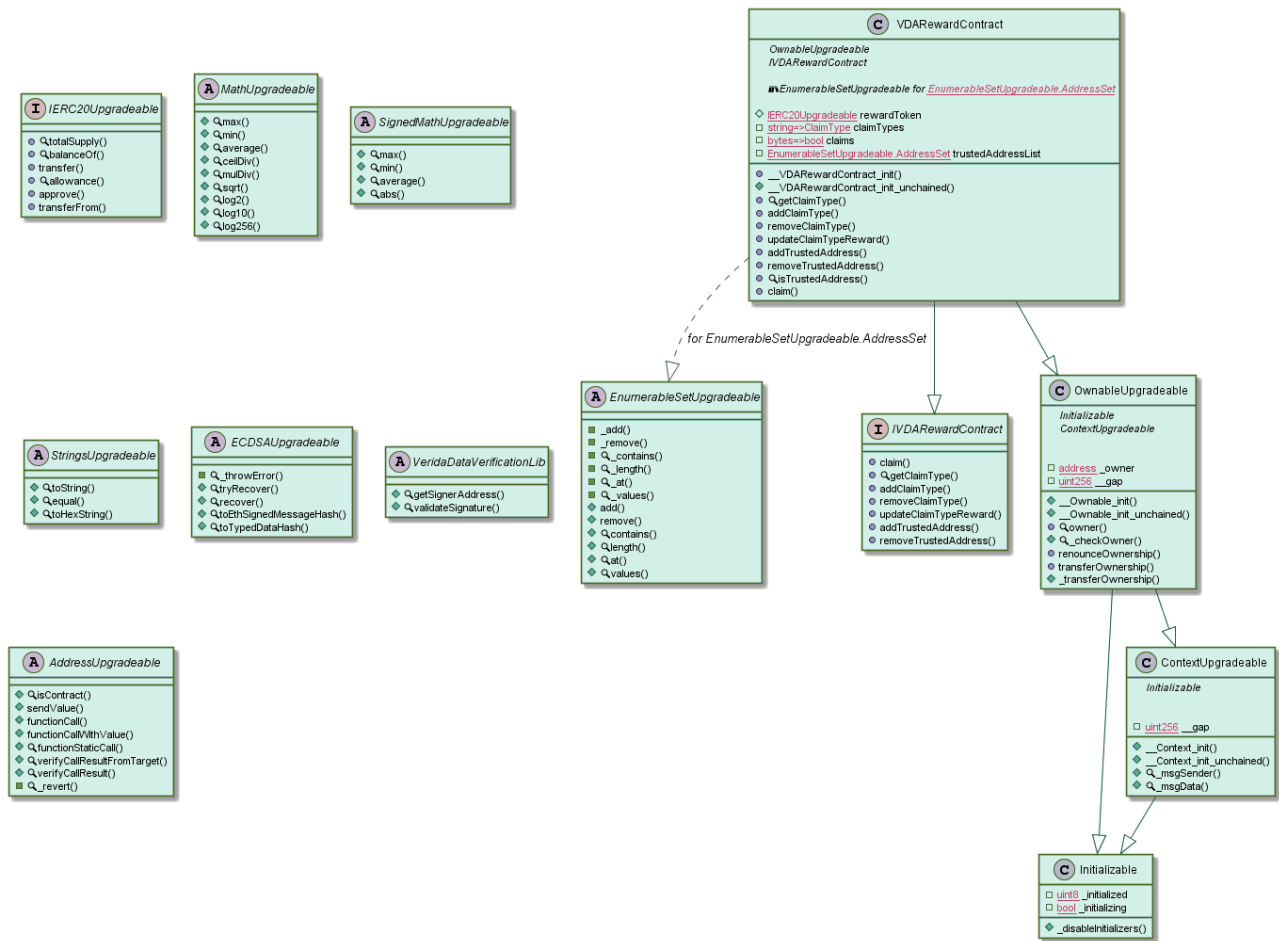
NameRegistry Diagram



#Hashlock.



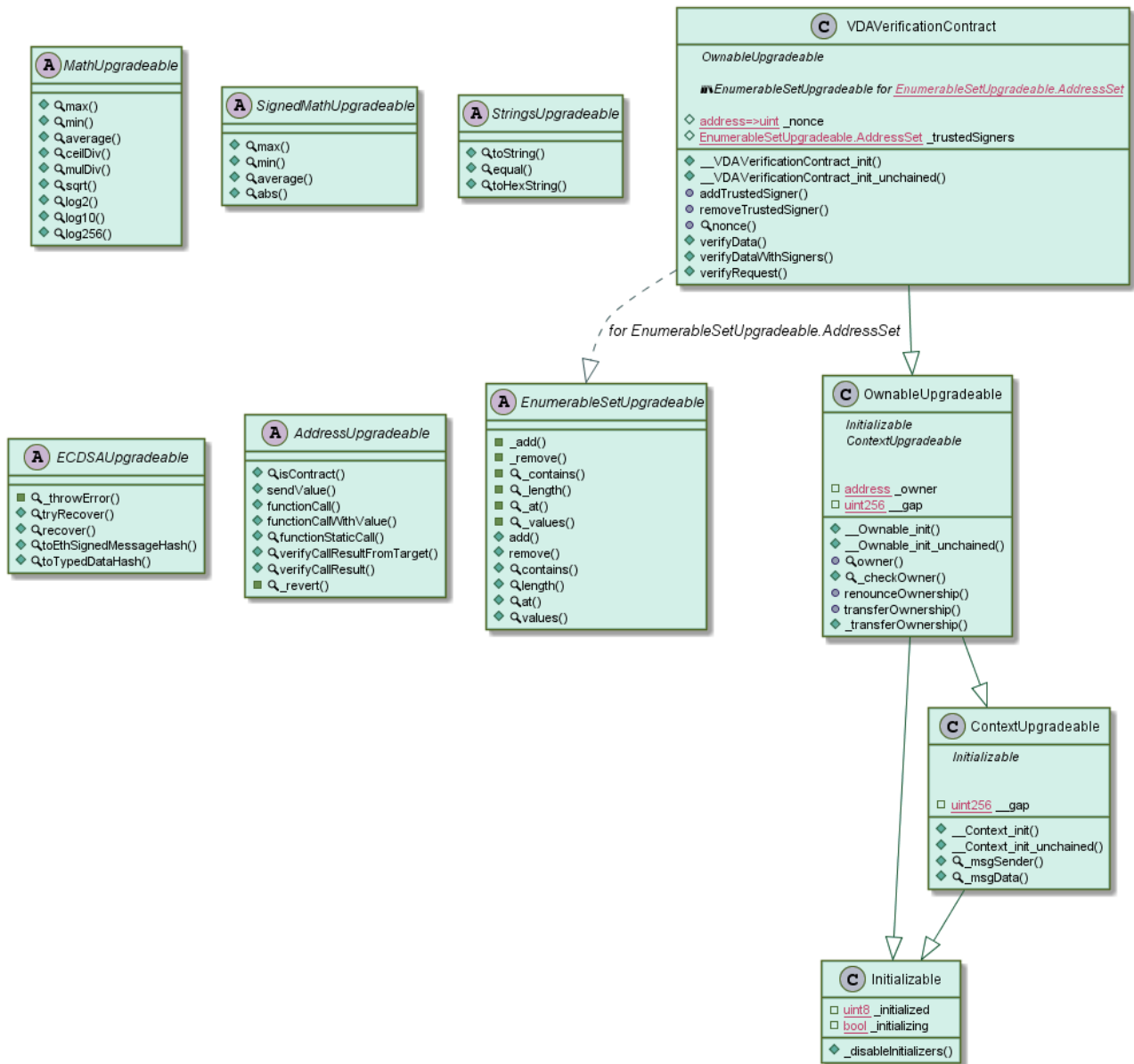
VDARewardContract Diagram



#Hashlock.



VDAVerificationContract Diagram



Slither Results Log

Slither log >> VeridaDIDLinkage.sol

```
MathUpgradeable.mulDiv(uint256,uint256,uint256) (VeridaDIDLinkage.sol#50-126) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#57-61)
- INLINE ASM (VeridaDIDLinkage.sol#77-84)
- INLINE ASM (VeridaDIDLinkage.sol#91-100)
StringsUpgradeable.toString(uint256) (VeridaDIDLinkage.sol#375-395) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#381-383)
- INLINE ASM (VeridaDIDLinkage.sol#387-389)
EnumerableSetUpgradeable.values(EnumerableSetUpgradeable.Bytes32Set) (VeridaDIDLinkage.sol#808-818) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#813-815)
EnumerableSetUpgradeable.values(EnumerableSetUpgradeable.AddressSet) (VeridaDIDLinkage.sol#882-892) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#887-889)
EnumerableSetUpgradeable.values(EnumerableSetUpgradeable.UintSet) (VeridaDIDLinkage.sol#956-966) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#961-963)
ECDSAUpgradeable.tryRecover(bytes32,bytes) (VeridaDIDLinkage.sol#1010-1027) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#1018-1022)
ECDSAUpgradeable.toEthSignedMessageHash(bytes32) (VeridaDIDLinkage.sol#1120-1129) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#1124-1128)
ECDSAUpgradeable.toTypedDataHash(bytes32,bytes32) (VeridaDIDLinkage.sol#1152-1161) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#1154-1160)
AddressUpgradeable._revert(bytes,string) (VeridaDIDLinkage.sol#1249-1258) uses assembly
- INLINE ASM (VeridaDIDLinkage.sol#1251-1254)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
```

```
Pragma version^0.8.4 (VeridaDIDLinkage.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
```

```
Low level call in AddressUpgradeable.sendValue(address,uint256) (VeridaDIDLinkage.sol#1170-1175):
- (success) = recipient.call{value: amount}() (VeridaDIDLinkage.sol#1173)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (VeridaDIDLinkage.sol#1197-1206):
- (success,returndata) = target.call{value: value}(data) (VeridaDIDLinkage.sol#1204)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (VeridaDIDLinkage.sol#1212-1219):
- (success,returndata) = target.staticcall(data) (VeridaDIDLinkage.sol#1217)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
```

```
Function ContextUpgradeable.__Context_init() (VeridaDIDLinkage.sol#1309-1310) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (VeridaDIDLinkage.sol#1312-1313) is not in mixedCase
Variable ContextUpgradeable.__gap (VeridaDIDLinkage.sol#1322) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (VeridaDIDLinkage.sol#1333-1335) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (VeridaDIDLinkage.sol#1337-1339) is not in mixedCase
Variable OwnableUpgradeable.__gap (VeridaDIDLinkage.sol#1398) is not in mixedCase
Function VDAVerificationContract.__VDAVerificationContract_init() (VeridaDIDLinkage.sol#1415-1418) is not in mixedCase
Function VDAVerificationContract.__VDAVerificationContract_init_unchained() (VeridaDIDLinkage.sol#1424-1425) is not in mixedCase
Variable VDAVerificationContract.__nonce (VeridaDIDLinkage.sol#1406) is not in mixedCase
Variable VDAVerificationContract.trustedSigners (VeridaDIDLinkage.sol#1409) is not in mixedCase
Parameter VeridaDIDLinkage.parseAddr(string).a (VeridaDIDLinkage.sol#1764) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
VeridaDIDLinkage.sol analyzed (13 contracts with 84 detectors), 94 result(s) found
```

Slither log >> VeridaDIDRegistry.sol

```
Pragma version^0.8.4 (VeridaDIDRegistry.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
```

```
Low level call in AddressUpgradeable.sendValue(address,uint256) (VeridaDIDRegistry.sol#1175-1180):
- (success) = recipient.call{value: amount}() (VeridaDIDRegistry.sol#1178)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (VeridaDIDRegistry.sol#1202-1211):
- (success,returndata) = target.call{value: value}(data) (VeridaDIDRegistry.sol#1209)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (VeridaDIDRegistry.sol#1217-1224):
- (success,returndata) = target.staticcall(data) (VeridaDIDRegistry.sol#1222)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
```

```
Parameter VeridaDataVerificationLib.getSignerAddress(bytes,bytes)._unsignedData (VeridaDIDRegistry.sol#642) is not in mixedCase
Parameter VeridaDataVerificationLib.getSignerAddress(bytes,bytes)._signature (VeridaDIDRegistry.sol#643) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._unsignedData (VeridaDIDRegistry.sol#650) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._signature (VeridaDIDRegistry.sol#651) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._signerAddress (VeridaDIDRegistry.sol#652) is not in mixedCase
Function ContextUpgradeable.__Context_init() (VeridaDIDRegistry.sol#1314-1315) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (VeridaDIDRegistry.sol#1317-1318) is not in mixedCase
Variable ContextUpgradeable.__gap (VeridaDIDRegistry.sol#1327) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (VeridaDIDRegistry.sol#1338-1340) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (VeridaDIDRegistry.sol#1342-1344) is not in mixedCase
Variable OwnableUpgradeable.__gap (VeridaDIDRegistry.sol#1403) is not in mixedCase
Variable VeridaDIDRegistry._DIDInfo (VeridaDIDRegistry.sol#1421) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
VeridaDIDRegistry.sol analyzed (13 contracts with 84 detectors), 97 result(s) found
```

Slither log >> NameRegistry.sol

```

Pragma version^0.8.4 (NameRegistry.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (NameRegistry.sol#2354-2359):
- (success) = recipient.call{value: amount}() (NameRegistry.sol#2357)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (NameRegistry.sol#2381-2390):
- (success,returndata) = target.call{value: value}(data) (NameRegistry.sol#2388)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (NameRegistry.sol#2396-2403):
- (success,returndata) = target.staticcall(data) (NameRegistry.sol#2401)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Contract console (NameRegistry.sol#5-1533) is not in CapWords
Parameter VeridaDataVerificationLib.getSignerAddress(bytes,bytes)._unsignedData (NameRegistry.sol#2172) is not in mixedCase
Parameter VeridaDataVerificationLib.getSignerAddress(bytes,bytes)._signature (NameRegistry.sol#2173) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._unsignedData (NameRegistry.sol#2180) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._signature (NameRegistry.sol#2181) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._signerAddress (NameRegistry.sol#2182) is not in mixedCase
Parameter StringLib.upper(string)._base (NameRegistry.sol#2271) is not in mixedCase
Parameter StringLib.lower(string)._base (NameRegistry.sol#2292) is not in mixedCase
Function ContextUpgradeable.__Context_init() (NameRegistry.sol#2493-2494) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (NameRegistry.sol#2496-2497) is not in mixedCase
Variable ContextUpgradeable.__gap (NameRegistry.sol#2506) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (NameRegistry.sol#2517-2519) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (NameRegistry.sol#2521-2523) is not in mixedCase
Variable OwnableUpgradeable.__gap (NameRegistry.sol#2582) is not in mixedCase
Variable NameRegistry._nonce (NameRegistry.sol#2594) is not in mixedCase
Variable NameRegistry._DIDInfoList (NameRegistry.sol#2614) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
NameRegistry.sol analyzed (13 contracts with 84 detectors), 469 result(s) found

```

Slither log >> RewardToken.sol

```

Pragma version^0.8.4 (RewardToken.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (RewardToken.sol#39-44):
- (success) = recipient.call{value: amount}() (RewardToken.sol#42)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (RewardToken.sol#66-75):
- (success,returndata) = target.call{value: value}(data) (RewardToken.sol#73)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (RewardToken.sol#81-88):
- (success,returndata) = target.staticcall(data) (RewardToken.sol#86)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Function ContextUpgradeable.__Context_init() (RewardToken.sol#178-179) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (RewardToken.sol#181-182) is not in mixedCase
Variable ContextUpgradeable.__gap (RewardToken.sol#191) is not in mixedCase
Function ERC20Upgradeable.__ERC20_init(string,string) (RewardToken.sol#205-207) is not in mixedCase
Function ERC20Upgradeable.__ERC20_init_unchained(string,string) (RewardToken.sol#209-212) is not in mixedCase
Variable ERC20Upgradeable.__gap (RewardToken.sol#369) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (RewardToken.sol#381-383) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (RewardToken.sol#385-387) is not in mixedCase
Variable OwnableUpgradeable.__gap (RewardToken.sol#446) is not in mixedCase
Function RewardToken._RewardToken_init() (RewardToken.sol#459-463) is not in mixedCase
Function RewardToken._RewardToken_init_unchained() (RewardToken.sol#465-466) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
RewardToken.sol analyzed (8 contracts with 84 detectors), 32 result(s) found

```

Slither log >> VDARewardContract.sol

```

Pragma version^0.8.4 (VDARewardContract.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (VDARewardContract.sol#1129-1134):
- (success) = recipient.call{value: amount}() (VDARewardContract.sol#1132)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (VDARewardContract.sol#1156-1165):
- (success,returndata) = target.call{value: value}(data) (VDARewardContract.sol#1163)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (VDARewardContract.sol#1171-1178):
- (success,returndata) = target.staticcall(data) (VDARewardContract.sol#1176)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Parameter VeridaDataVerificationLib.getSignerAddress(bytes,bytes)._unsignedData (VDARewardContract.sol#1004) is not in mixedCase
Parameter VeridaDataVerificationLib.getSignerAddress(bytes,bytes)._signature (VDARewardContract.sol#1005) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._unsignedData (VDARewardContract.sol#1012) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._signature (VDARewardContract.sol#1013) is not in mixedCase
Parameter VeridaDataVerificationLib.validateSignature(bytes,bytes,address)._signerAddress (VDARewardContract.sol#1014) is not in mixedCase
Function ContextUpgradeable.__Context_init() (VDARewardContract.sol#1268-1269) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (VDARewardContract.sol#1271-1272) is not in mixedCase
Variable ContextUpgradeable.__gap (VDARewardContract.sol#1281) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (VDARewardContract.sol#1292-1294) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (VDARewardContract.sol#1296-1298) is not in mixedCase
Variable OwnableUpgradeable.__gap (VDARewardContract.sol#1357) is not in mixedCase
Function VDARewardContract.__VDARewardContract_init(IERC20Upgradeable) (VDARewardContract.sol#1382-1385) is not in mixedCase
Function VDARewardContract.__VDARewardContract_init_unchained(IERC20Upgradeable) (VDARewardContract.sol#1387-1389) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
VDARewardContract.sol analyzed (13 contracts with 84 detectors), 101 result(s) found

```

Slither log >> SoulboundNFT.sol

```
Reentrancy in SoulboundNFT.claimSBT(address,ISoulboundNFT.SBTInfo,bytes,bytes) (SoulboundNFT.sol#4115-4174):
  External calls:
    - _safeMint(sbtInfo.recipient,tokenId) (SoulboundNFT.sol#4152)
      - IERC721ReceiverUpgradeable(to).onERC721Received(_msgSender(),from,tokenId,data) (SoulboundNFT.sol#2637-2648)
  State variables written after the call(s):
    - _tokenIdInfo[tokenId].sbtType = sbtInfo.sbtType (SoulboundNFT.sol#4159)
    - _tokenIdInfo[tokenId].uniqueId = sbtInfo.uniqueId (SoulboundNFT.sol#4160)
    - _setTokenURI(tokenId,sbtInfo.sbtURI) (SoulboundNFT.sol#4155)
      - _tokenURIs[tokenId] = _tokenURI (SoulboundNFT.sol#2771)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2

Reentrancy in SoulboundNFT.claimSBT(address,ISoulboundNFT.SBTInfo,bytes,bytes) (SoulboundNFT.sol#4115-4174):
  External calls:
    - _safeMint(sbtInfo.recipient,tokenId) (SoulboundNFT.sol#4152)
      - IERC721ReceiverUpgradeable(to).onERC721Received(_msgSender(),from,tokenId,data) (SoulboundNFT.sol#2637-2648)
  Event emitted after the call(s):
    - PermanentURI(sbtInfo.sbtURI,tokenId) (SoulboundNFT.sol#4171)
    - SBTClaimed(sbtInfo.recipient,tokenId,sbtInfo.sbtType) (SoulboundNFT.sol#4162)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

console._sendLogPayload(bytes) (SoulboundNFT.sol#8-15) uses assembly
  - INLINE ASM (SoulboundNFT.sol#11-14)
```

```
Pragma version^0.8.4 (SoulboundNFT.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
```

```
Low level call in AddressUpgradeable.sendValue(address,uint256) (SoulboundNFT.sol#1857-1862):
  - (success) = recipient.call{value: amount}() (SoulboundNFT.sol#1860)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (SoulboundNFT.sol#1884-1893):
  - (success,returndata) = target.call{value: value}(data) (SoulboundNFT.sol#1891)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (SoulboundNFT.sol#1899-1906):
  - (success,returndata) = target.staticcall(data) (SoulboundNFT.sol#1904)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
```

```
Contract console (SoulboundNFT.sol#5-1533) is not in CapWords
Function ContextUpgradeable.__Context_init() (SoulboundNFT.sol#1996-1997) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (SoulboundNFT.sol#1999-2000) is not in mixedCase
Variable ContextUpgradeable.__gap (SoulboundNFT.sol#2009) is not in mixedCase
Function ERC165Upgradeable.__ERC165_init() (SoulboundNFT.sol#2196-2197) is not in mixedCase
Function ERC165Upgradeable.__ERC165_init_unchained() (SoulboundNFT.sol#2199-2200) is not in mixedCase
Variable ERC165Upgradeable.__gap (SoulboundNFT.sol#2213) is not in mixedCase
Function ERC721Upgradeable.__ERC721_init(string,string) (SoulboundNFT.sol#2241-2243) is not in mixedCase
Function ERC721Upgradeable.__ERC721_init_unchained(string,string) (SoulboundNFT.sol#2245-2248) is not in mixedCase
Variable ERC721Upgradeable.__gap (SoulboundNFT.sol#2728) is not in mixedCase
Function ERC721URIStorageUpgradeable.__ERC721URIStorage_init() (SoulboundNFT.sol#2731-2732) is not in mixedCase
Function ERC721URIStorageUpgradeable.__ERC721URIStorage_init_unchained() (SoulboundNFT.sol#2734-2735) is not in mixedCase
Variable ERC721URIStorageUpgradeable.__gap (SoulboundNFT.sol#2793) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (SoulboundNFT.sol#3703-3705) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (SoulboundNFT.sol#3707-3709) is not in mixedCase
Variable OwnableUpgradeable.__gap (SoulboundNFT.sol#3768) is not in mixedCase
Function VDAVerificationContract.__VDAVerificationContract_init() (SoulboundNFT.sol#3785-3788) is not in mixedCase
Function VDAVerificationContract.__VDAVerificationContract_init_unchained() (SoulboundNFT.sol#3794-3795) is not in mixedCase
Variable VDAVerificationContract.__nonce (SoulboundNFT.sol#3776) is not in mixedCase
Variable VDAVerificationContract.__trustedSigners (SoulboundNFT.sol#3779) is not in mixedCase
Variable SoulboundNFT.userTokenIds (SoulboundNFT.sol#3992) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
SoulboundNFT.sol analyzed (23 contracts with 84 detectors), 498 result(s) found
```

Slither log >> VDAVerificationContract.sol

```
Pragma version^0.8.4 (VDAVerificationContract.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in AddressUpgradeable.sendValue(address,uint256) (VDAVerificationContract.sol#983-988):
  - (success) = recipient.call{value: amount}() (VDAVerificationContract.sol#986)
Low level call in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string) (VDAVerificationContract.sol#1010-1019):
  - (success,returndata) = target.call{value: value}(data) (VDAVerificationContract.sol#1017)
Low level call in AddressUpgradeable.functionStaticCall(address,bytes,string) (VDAVerificationContract.sol#1025-1032):
  - (success,returndata) = target.staticcall(data) (VDAVerificationContract.sol#1030)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Function ContextUpgradeable.__Context_init() (VDAVerificationContract.sol#1122-1123) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (VDAVerificationContract.sol#1125-1126) is not in mixedCase
Variable ContextUpgradeable.__gap (VDAVerificationContract.sol#1135) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init() (VDAVerificationContract.sol#1146-1148) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained() (VDAVerificationContract.sol#1150-1152) is not in mixedCase
Variable OwnableUpgradeable.__gap (VDAVerificationContract.sol#1211) is not in mixedCase
Function VDAVerificationContract.__VDAVerificationContract_init() (VDAVerificationContract.sol#1228-1231) is not in mixedCase
Function VDAVerificationContract.__VDAVerificationContract_init_unchained() (VDAVerificationContract.sol#1237-1238) is not in mixedCase
Variable VDAVerificationContract.__nonce (VDAVerificationContract.sol#1219) is not in mixedCase
Variable VDAVerificationContract.__trustedSigners (VDAVerificationContract.sol#1222) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
VDAVerificationContract.sol analyzed (10 contracts with 84 detectors), 108 result(s) found
```

Solidity Static Analysis

VeridaDIDLinkage.sol

Security

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 1173:34:

Gas & Economy

Gas costs:

Gas requirement of function VeridaDIDLinkage.getLinks is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 1747:11:

Delete dynamic array:

The "delete" operation when applied to a dynamically sized array in Solidity generates code to delete each of the elements contained. If the array is large, this operation can surpass the block gas limit and raise an OOG exception. Also nested dynamically sized objects can produce the same results.

[more](#)

Pos: 1724:15:

Miscellaneous

No return:

IVeridaDIDLinkage.lookup(string): Defines a return type but never explicitly returns a value.

Pos: 581:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 1818:15:

Delete from dynamic array:

Using "delete" on an array leaves a gap. The length of the array remains the same. If you want to remove the empty position you need to shift items manually and update the "length" property.

[more](#)

Pos: 1724:15:

VeridaDIDRegistry.sol

Security

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 1256:19:

Gas & Economy

Gas costs:

Gas requirement of function VeridaDIDRegistry.register is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 1455:9:

For loop over dynamic array:

Loops that do not have a fixed number of iterations, for example, loops that depend on storage values, have to be used carefully. Due to the block gas limit, transactions can only consume a certain amount of gas. The number of iterations in a loop can grow beyond the block gas limit which can cause the complete contract to be stalled at a certain point. Additionally, using unbounded loops incurs in a lot of avoidable gas costs. Carefully test how many items at maximum you can pass to such functions to make it successful.

[more](#)

Pos: 1462:13:

Miscellaneous

Delete from dynamic array:

Using "delete" on an array leaves a gap. The length of the array remains the same. If you want to remove the empty position you need to shift items manually and update the "length" property.

[more](#)

Pos: 1058:23:

NameRegistry.sol

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in `AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string)`: Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 2381:11:

Gas & Economy

Gas costs:

Gas requirement of function NameRegistry.getUserNameList is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 2731:11:

Miscellaneous**Guard conditions:**

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 2735:15:

RewardToken.sol**Security****Check-effects-interaction:**

Potential violation of Checks-Effects-Interaction pattern in AddressUpgradeable.functionCallWithValue(address,bytes,uint256,string): Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 66:4:

Gas & Economy**Gas costs:**

Gas requirement of function RewardToken.mint is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 472:4:

Miscellaneous

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 473:8:

VDARewardContract.sol

Security

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 1163:57:

Gas & Economy

Gas costs:

Gas requirement of function VDARewardContract.claim is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 1466:11:

Miscellaneous

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 1481:15:

Delete from dynamic array:

Using "delete" on an array leaves a gap. The length of the array remains the same. If you want to remove the empty position you need to shift items manually and update the "length" property.

[more](#)

Pos: 1419:15:

SoulboundNFT.sol

Security

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 3684:15:

Gas & Economy

Gas costs:

Gas requirement of function SoulboundNFT.initialize is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 4002:11:

Gas costs:

Gas requirement of function SoulboundNFT.getClaimedSBTList is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 4179:11:

Miscellaneous

Constant/View/Pure functions:

`EnumerableSetUpgradeable.values(struct EnumerableSetUpgradeable.UintSet)` :
Is constant but potentially should not be. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 3122:4:

Data truncated:

Division of integer values yields an integer value again. That means e.g. $10 / 100 = 0$ instead of 0.1 since the result is an integer again. This does not hold for division of (only) literal values since those yield rational constants.

Pos: 3304:35:

VDAVerificationContract.sol**Security****Low level calls:**

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 1017:57:

Gas & Economy**Gas costs:**

Gas requirement of function `VDAVerificationContract.addTrustedSigner` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 1245:11:

Miscellaneous**Guard conditions:**

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 1355:15:

Solhint Linter

VeridaDIDLinkage.sol

```
VeridaDIDLinkage.sol:51:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:165:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:181:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:193:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:234:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:246:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:283:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:297:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:326:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:361:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:376:18: Error: Parse error: missing ';' at '{'  
VeridaDIDLinkage.sol:408:18: Error: Parse error: missing ';' at '{'
```

VeridaDIDRegistry.sol

```
VeridaDIDRegistry.sol:52:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:166:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:182:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:194:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:235:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:247:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:284:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:298:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:327:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:362:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:377:18: Error: Parse error: missing ';' at '{'  
VeridaDIDRegistry.sol:409:18: Error: Parse error: missing ';' at '{'
```

NameRegistry.sol

```
NameRegistry.sol:1582:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1696:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1712:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1724:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1765:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1777:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1814:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1828:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1857:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1892:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1907:18: Error: Parse error: missing ';' at '{'  
NameRegistry.sol:1939:18: Error: Parse error: missing ';' at '{'
```

RewardToken.sol

```
RewardToken.sol:271:18: Error: Parse error: missing ';' at '{'  
RewardToken.sol:290:18: Error: Parse error: missing ';' at '{'  
RewardToken.sol:306:18: Error: Parse error: missing ';' at '{'  
RewardToken.sol:321:18: Error: Parse error: missing ';' at '{'  
RewardToken.sol:351:22: Error: Parse error: missing ';' at '{'
```

VDARewardContract.sol

```
VDARewardContract.sol:75:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:189:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:205:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:217:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:258:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:270:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:307:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:321:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:350:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:385:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:400:18: Error: Parse error: missing ';' at '{'  
VDARewardContract.sol:432:18: Error: Parse error: missing ';' at '{'
```

SoulboundNFT.sol

```
SoulboundNFT.sol:1548:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:1556:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:2496:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:2533:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:2572:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3182:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3296:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3312:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3324:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3365:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3377:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3414:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3428:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3457:18: Error: Parse error: missing ';' at '{'  
SoulboundNFT.sol:3492:18: Error: Parse error: missing ';' at '{'
```

VDAVerificationContract.sol

```
VDAVerificationContract.sol:390:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:504:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:520:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:532:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:573:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:585:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:622:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:636:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:665:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:700:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:715:18: Error: Parse error: missing ';' at '{'  
VDAVerificationContract.sol:747:18: Error: Parse error: missing ';' at '{'
```

Software analysis result:

This software reported many false positive results and some informational issues.

These issues can be safely ignored.

#Hashlock.

